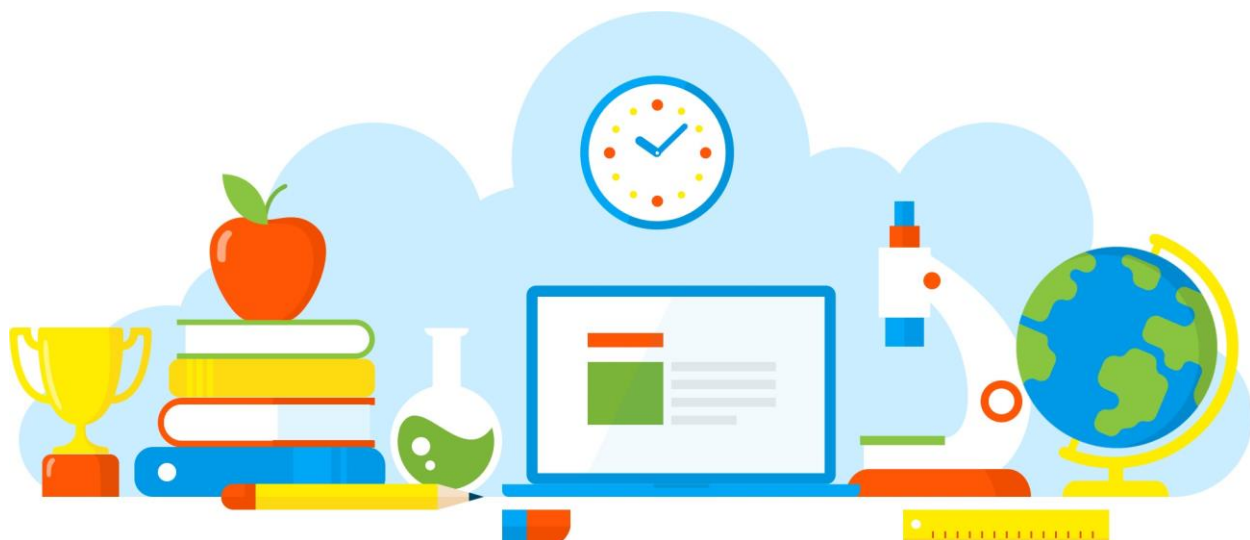




Barra de Navegación

Parte 2

[Gabriel Barrón Rodríguez](#)

Introducción

En este tutorial, vamos a implementar los componentes que realiza la barra del menú de navegación

 EL SITIO DE BARG **CURSOS**

[Inicio](#)

[Acerca de](#)

[Cursos](#)



[Ingresar](#)

Bienvenido a **BARG Cursos**

Instalación de módulo

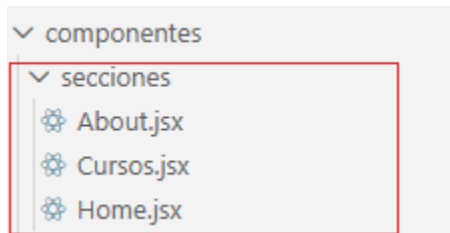
El módulo **React-Router-Dom** es fundamental si quieres crear una aplicación web moderna (una Single Page Application o SPA) con React.

- ✓ Instalar el módulo React-Router-Dom

```
npm install react-router-dom
```

Creación de los componentes Home, About y Cursos

Crear una carpeta llamada secciones dentro de la carpeta componentes



Dentro de la carpeta **Secciones** crear los componentes About, Cursos y Home

Componente Home

El siguiente código representa una sección de bienvenida escrita en **React** utilizando **Tailwind CSS** para los estilos.

```
const Home = () => {
  return (
    <section
      id="/"
      className="h-screen flex items-center justify-center bg-gray-50"
    >
      <h1 className="text-5xl font-bold">
        Bienvenido a <span className="text-primary">BARG Cursos</span>
      </h1>
    </section>
  );
};

export default Home;
```

Componente About

Esta sección está diseñada para ser el bloque de "**Sobre Nosotros**" (About Us). A diferencia de la anterior, esta no ocupa toda la pantalla, sino que tiene un tamaño basado en su contenido con márgenes internos.

```
const About = () => {
  return (
    <section id="#about" className="py-20 bg-white">
      <div className="container">
        <h2 className="text-3xl font-bold mb-8">Sobre Nosotros</h2>
        <p className="text-lg">Aquí iría la información sobre la empresa o institución.</p>
      </div>
    </section>
  )
}

export default About
```

Componente Cursos

Esta sección es una **cuadrícula de servicios o productos** (en este caso, cursos). Lo más importante aquí es el uso de **CSS Grid**, que permite organizar las tarjetas de forma automática según el tamaño de la pantalla.

```
const Cursos = () => {
  return (
    <section id="#cursos" className="py-20 bg-white">
      <div className="container">
        <h2 className="text-3xl font-bold mb-8">Nuestros Cursos</h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
          /* Aquí irían tus tarjetas de cursos */
          <div className="p-6 shadow-lg rounded-xl bg-orange-50">Curso de React</div>
          <div className="p-6 shadow-lg rounded-xl bg-orange-50">Curso de Tailwind</div>
          <div className="p-6 shadow-lg rounded-xl bg-orange-50">Curso de Framer
            Motion</div>
        </div>
      </div>
    </section>
  )
}

export default Cursos
```

Es necesario modificar las rutas en el archivo [data.js](#), para que coincidan hacia los componentes a desarrollar

```
1  const NavbarLinks = [
2    {
3      id: 1,
4      name: "Home",
5      title: "Inicio",
6      url: "/",
7    },
8    {
9      id: 2,
10     name: "About",
11     title: "Acerca de",
12     url: "/about",
13   },
14   {
15     id: 3,
16     name: "courses",
17     title: "Cursos",
18     url: "/cursos",
19   }
20 ];
```

Configurar `<BrowserRouter>` es necesario porque actúa como el cerebro y motor de la navegación en una aplicación de React. Sin él, los componentes que usaste antes (como las secciones de "Cursos" o "Nosotros") no podrían conectarse a la URL del navegador.

Configurar el archivo App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "../componentes/secciones/Home.jsx";
import Cursos from "../componentes/secciones/Cursos.jsx";
import About from "../componentes/secciones/About.jsx";
import Navbar from "../componentes/Navbar.jsx";
```

```
const App = () => {
  return (
    <div className="overflow-x-hidden">
      <BrowserRouter>
        <Navbar />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/cursos" element={<Cursos />} />
          <Route path="/about" element={<About />} />
        </Routes>
      </BrowserRouter>
    </div>
  );
};
```

```
export default App;
```

Pull Request Github

Un Pull Request (PR) es una petición de revisión. Es la forma en que un desarrollador le dice a sus compañeros: "He terminado estos cambios en mi propia rama, ¿podrían revisarlos e integrarlos en el proyecto principal?".

Es la herramienta central para el trabajo colaborativo en GitHub.

Instrucciones

- ✓ Para continuar con el ejercicio se recomienda realizar una copia del proyecto Barra de Navegación parte 1 para seguir con esta parte 2.
- ✓ Verificar que se está trabajando con la rama main o en caso contrario cambiar al nombre de la rama main

```
git branch -M main
```

- ✓ Verificar que Git está configurada con cuenta de GitHub en caso de que no exista dicha configuración

```
git config --global user.email "correo electronico"  
git config --global user.name "usuario en Github"
```

- ✓ Generar una clave SSH (en caso de que no la tengas) donde se recomienda el algoritmo **Ed25519** por ser más seguro y eficiente que el antiguo RSA.

```
ssh-keygen -t ed25519 -C "tu_email@ejemplo.com"
```

- ✓ Al terminar, se generan dos archivos en la carpeta c:/usuarios/./.ssh/. Es vital que el estudiante entienda la diferencia:

id_ed25519: Clave Privada. Es como tu llave física. Nunca se comparte.

id_ed25519.pub: Clave Pública. Es como el candado. Esta es la que se sube a GitHub.

- ✓ Para conectar con GitHub/GitLab, el estudiante debe copiar el contenido del archivo **público** (.pub).

```
C:\Users\Usuario\.ssh\ id_ed25519.pub
```

- ✓ Configuración en GitHub
 - Hacer clic en su foto de perfil (esquina superior derecha).
 - Ir a Settings (Configuración).
 - En el menú lateral izquierdo, buscar la sección "Access" y hacer clic en SSH and GPG keys.
 - Hacer clic en el botón verde New SSH key.
 - Title: Poner un nombre descriptivo (ej: "Laptop de Juan" o "PC Laboratorio").
 - Key type: Dejarlo como "Authentication Key".
 - Key: Pegar el código que copiaron en el paso anterior.
 - Hacer clic en Add SSH key.
- ✓ Muchos estudiantes clonan sus repositorios por HTTPS por error, cambiar el origen a SSH para que la clave funcione:

Para ver si están usando HTTPS o SSH

`git remote -v`

Establecer la URL

`git remote set-url origin git@github.com:Frameworks-usuario.git`

- ✓ Probar que está correcto

```
C:\Users\Usuario\Documents\ReactProyectos\FrameWorks Web\gtie157_navbar>ssh -T git@github.com
Enter passphrase for key 'C:\Users\Usuario\.ssh/id_ed25519':
Hi gbarron2014! You've successfully authenticated, but GitHub does not provide shell access.
```

Pull Request

- ✓ Ubicarse en la carpeta raíz del proyecto e inicializar repositorio a través del comando
- `git init`
- ✓ Del repositorio que has creado para la barra de navegación cerciórate apuntar a la URL del repositorio

-
- ✓ Crear una rama en específico para la funcionalidad

`git checkout -b feature/barra_navegacion`

- `git checkout`: Es la instrucción para cambiar de una rama a otra.
- `-b`: Es la bandera (flag) de "branch". Le dice a Git: "Oye, esta rama no existe todavía, créala antes de intentar cambiarme a ella".
- `feature/barra_navegacion`: Es el nombre de la rama.

- ✓ Agregar los cambios a través del siguiente comando

`git add .`

- ✓ Guardar los cambios en el estado local con el comentario correspondiente,

`git commit -m "Barra de Navegacion"`

`git commit`: Le indica a Git que quieres consolidar los cambios que previamente agregaste con `git add`.

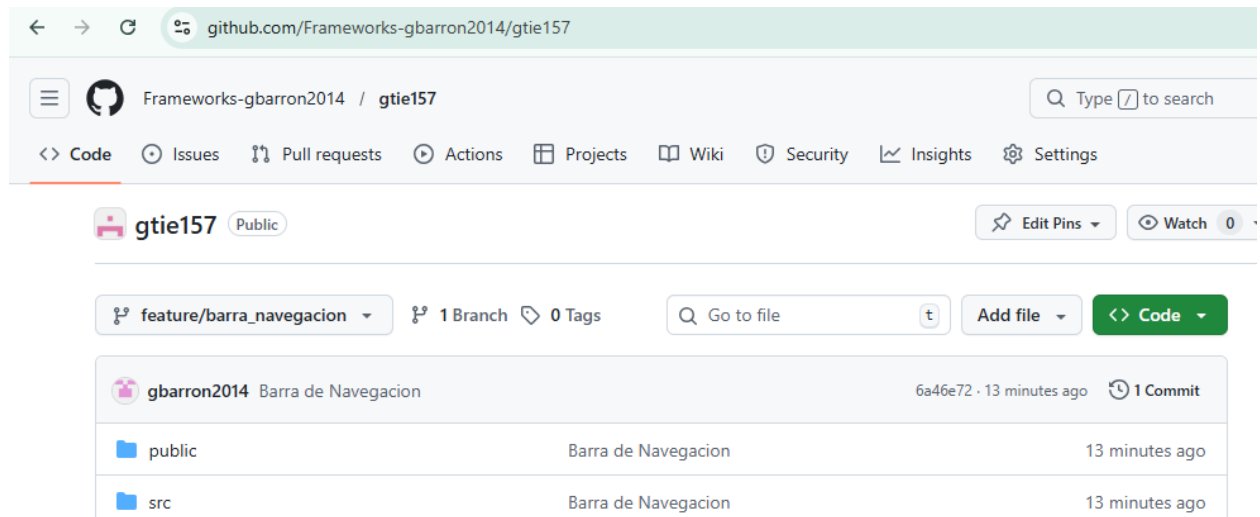
`-m`: Es la abreviatura de "message" (mensaje). Indica que el texto que sigue entre comillas es la descripción de lo que hiciste.

"Barra de Navegacion": Es el comentario que verán tus compañeros (o tú mismo en el futuro).

- ✓ Subir los cambios con el comando, toma en cuenta el nombre de la rama

`git push origin feature/barra_navegacion`

✓ Verificar qué los cambios hayan resultado



Desafío

- Personaliza los componentes Home, About y Curso
- Agregar dos opciones más en el menú
- Agregar opción de Login como Interfaz.
- Clonar el repositorio para que no pierdas los cambios, personalizar el contenido de los componentes de acuerdo a tu idea de proyecto de trabajo. Crear una rama feature/login y realiza los pull_request
- Crear imagen en Docker y ejecutarla